
doppler_dimming_lib

Release 1.0

Hervé Haudemand

May 07, 2026

CONTENTS:

1	Introduction	1
1.1	doppler_dimming_lib package	1
1.1.1	Subpackages	1
1.1.1.1	doppler_dimming_lib.c_libraries package	1
1.1.1.1.1	Submodules	1
1.1.1.1.2	doppler_dimming_lib.c_libraries.c_lib module	1
1.1.1.1.3	doppler_dimming_lib.c_libraries.integrands_lib module	1
1.1.1.1.4	Module contents	1
1.1.1.2	doppler_dimming_lib.data package	1
1.1.1.2.1	Module contents	1
1.1.1.3	doppler_dimming_lib.spectra package	2
1.1.1.3.1	Submodules	2
1.1.1.3.2	doppler_dimming_lib.spectra.spectra module	2
1.1.1.3.3	Module contents	2
1.1.2	Submodules	2
1.1.3	doppler_dimming_lib.codex module	2
1.1.4	doppler_dimming_lib.datacubes module	5
1.1.5	doppler_dimming_lib.integrals module	5
1.1.6	doppler_dimming_lib.inversion module	6
1.1.7	doppler_dimming_lib.metis module	7
1.1.8	doppler_dimming_lib.parallelization module	9
1.1.9	doppler_dimming_lib.utils module	9
1.1.10	Module contents	10
2	Indices and tables	11
	Python Module Index	13
	Index	15

INTRODUCTION

This documentation covers the `doppler_dimming_lib` library to invert the electron temperature.

1.1 `doppler_dimming_lib` package

1.1.1 Subpackages

1.1.1.1 `doppler_dimming_lib.c_libraries` package

1.1.1.1.1 Submodules

1.1.1.1.2 `doppler_dimming_lib.c_libraries.c_lib` module

`doppler_dimming_lib.c_libraries.c_lib.get_c_library(path: str = None)`

Sets up the `c_library` object which is used by the library. This object contains C integrand functions.

Parameters

path (*str*, *optional*) – path to shared compiled library. If None, `__file__ + ../integrands_lib.so` is used. Defaults to None.

Returns

CDLL library containing integrand functions.

Return type

`ctypes.CDLL`

1.1.1.1.3 `doppler_dimming_lib.c_libraries.integrands_lib` module

1.1.1.1.4 Module contents

1.1.1.2 `doppler_dimming_lib.data` package

1.1.1.2.1 Module contents

Contains data

1.1.1.3 doppler_dimming_lib.spectra package

1.1.1.3.1 Submodules

1.1.1.3.2 doppler_dimming_lib.spectra.spectra module

`doppler_dimming_lib.spectra.spectra.get_spectrum_from_txt(precision_level: int = 0) → ndarray`

Returns the spectrum from txt file. Use the precision level to determine the sampling quality.

Parameters

precision_level (*int, optional*) – : 0 to 2 increasing spectrum sampling, sparse (0, Thekaekara), medium (1, Thuillier) or dense (2, MODTRAN). Defaults to 0 for faster computations.

Returns

spectrum wavelengths in [cm] and irradiances in [erg / s cm² sr Å].

Return type

np.ndarray

`doppler_dimming_lib.spectra.spectra.set_spectrum_level(spectrum_level: int = 0) → c_void_p`

Sets the spectrum to be used by this library.

Parameters

spectrum_level (*int, optional*) – spectrum to load, from sparser (0) to denser (2), see `get_spectrum_for_txt`. Defaults to 0.

Returns

pointer to spectra data, to be passed to c library

Return type

ctypes.c_void_p

1.1.1.3.3 Module contents

1.1.2 Submodules

1.1.3 doppler_dimming_lib.codex module

`doppler_dimming_lib.codex.convolve_codex_filter(_lambda: float, I, verbose: bool = True) → float`

Convolve a function I with the codex filter at `_lambda` transmissivity. For each available filter transmissivity T, computes the sum of

Parameters

- **_lambda** (*float*) – filter center in Angstrom.
- **I** (*callable*) – function to convolve, argument is `lambda` in Armstrong
- **verbose** (*bool, optional*) – print info about computation. Defaults to True.

Returns

convolution between `I(lambda)` and the filter transmissivity

Return type

float

doppler_dimming_lib.codex.get_R_as_spline(*integral_function*, *lambda1*: float, *lambda2*: float, ***kwargs*)

Find the cubic spline interpolator of the ratio function(T_1)/function(T_2). All the parameters of function must be provided as kwargs.

Parameters

- **function** (*callable*) – function to perform the ratio numerator/denominator
- **lambda1** (*float*) – numerator wavelength in angstrom
- **lambda2** (*float*) – denominator wavelength in angstrom

Returns

spline cubic interpolating $5.e5 < T < 3.e6$

Return type

PPoly

doppler_dimming_lib.codex.get_T_W_from_codex(*rho*: float, *Js*: list, *p0*: list) → list

Uses least squares to find the values of temperature and solar wind that better fit the input Js.

Parameters

- **rho** (*float*) – heliocentric distance in solar radii
- **Js** (*list*) – codex intensities at the four wavelengths, ordered
- **p0** (*list*) – initial guess for temperature and wind speed

Returns

fitted electron temperature and solar wind speed

Return type

list

doppler_dimming_lib.codex.get_T_from_R(*s2_over_t2*: float, *rho*: float, *wind_speed*: float = 0.0, ***kwargs*)

Retrieve the electron temperature from a CODEX measured ratio between filters S2 and T2. First gets R as spline for the selected rho, then gets T from the fitted spline.

Parameters

- **s2_over_t2** (*float*) – ratio between two filters S2/T2
- **rho** (*float*) – heliocentric distance in solar radii
- **wind_speed** (*float*, *optional*) – wind speed in km/s. Defaults to 0.

Returns

inferred electron temperature

Return type

uncertainties.ufloat

doppler_dimming_lib.codex.set_codex_transmittancies(*approx_filter*: bool = False, *skipevery*: int = 10, *min_transmittancy*: float = 0.001)

Reads the excel file containing the filter transmittancies and sets the dictionary containing arrays of wavelengths and transmittancies for each filter, ordered by center wavelength.

Parameters

- **approx_filter** (*bool*, *optional*) – approximate filter transmissivity with a box function. This may save some time, but results are often inconsistent. Defaults to False.
- **skipevery** (*int*, *optional*) – sample one transmittance every n entries. Defaults to 10.

- **min_transmittancy** (*float, optional*) – minimum transmittancy to consider, lower ones will be considered 0. Defaults to 0.001 (0.1%).

doppler_dimming_lib.codex.**simulate_codex_chunk**(*densities, z, r_pos, T_e_pos, wind_speed_pos, pid*)

doppler_dimming_lib.codex.**simulate_codex_filter**(*filter_name: str, rho: float, T_e: float, wind_speed: float, N_e_function=<function N_e_analytical>, verbose: bool = True*) → float

Simulate the measurement from a CODEX filter by convolving its response with Cram, 1976 integral

Parameters

- **filter_name** (*str*) – filter name, can be T1, S1, T2 or S2
- **rho** (*float*) – heliocentric distance in solar radii
- **T_e** (*float*) – electron temperature in Kelvin
- **wind_speed** (*float*) – wind speed in km/s
- **N_e_function** (*callable, optional*) – electron density as a function of the heliocentric distance (in rsun). Defaults to `utils.N_e_analytical`.

Returns

value measured with selected filter in erg cm⁻² s⁻¹ sr⁻¹

Return type

float

doppler_dimming_lib.codex.**simulate_codex_images**(*image_side, wind_speed_pos, densities_dc, coordinates_dc, T_e_pos, tot_processors=None*)

doppler_dimming_lib.codex.**simulate_codex_measure**(*rho: float, T_e: float, wind_speed: float, N_e_function=<function N_e_analytical>, verbose: bool = True*) → ndarray

Simulate the measurement from the four CODEX filters by convolving their response with Cram, 1976 integral

Parameters

- **rho** (*float*) – heliocentric distance in solar radii
- **T_e** (*float*) – electron temperature in Kelvin
- **wind_speed** (*float*) – wind speed in km/s
- **N_e_function** (*callable, optional*) – electron density as a function of the heliocentric distance (in rsun). Defaults to `utils.N_e_analytical`.

Returns

filters measured values, ordered by filter wavelength (T1=393, S1=398, T2=405, S2=423), in erg cm⁻² s⁻¹ sr⁻¹

Return type

np.ndarray(4)

1.1.4 doppler_dimming_lib.datacubes module

`doppler_dimming_lib.datacubes.datacube_from_map(_map, **kwargs)`

`doppler_dimming_lib.datacubes.map_from_ddt(filename: str, hdu_ID: int = 0, resample_to: list = None)`

1.1.5 doppler_dimming_lib.integrals module

`doppler_dimming_lib.integrals.I_dl_domega_dphi(rho: float, _lambda: float, T_e: float, W: float, verbose: bool = True) → float`

Interior integral, separated to allow calculations without integrating in dLOS

Parameters

- **rho** (*float*) – heliocentric distance in solar radii
- **_lambda** (*float*) – wavelength in angstrom
- **T_e** (*float*) – electron temperature in Kelvin
- **W** (*float*) – wind speed in km/s
- **verbose** (*bool, optional*) – print additional info at each step for debugging purposes. Defaults to True.

Returns

photospheric irradiance integrated over solar disc

Return type

float

`doppler_dimming_lib.integrals.I_s_lambda(*args, **kwargs) → float`

Wrapper for integrated_I_s used to compute total brightness by passing component=3 to integrated_I_s.

Parameters

- **rho** (*float*) – heliocentric distance in solar radii
- **_lambda** (*float*) – wavelength in angstrom
- **T_e** (*float*) – electron temperature
- **W** (*float*) – wind speed in km/s
- **min_x** (*float, optional*) – integration limit for LOS away from observer in solar radii. Defaults to 10.
- **max_x** (*float, optional*) – integration limit for LOS towards observer in solar radii. Defaults to 10.
- **N_e_function** (*callable, optional*) – function used to calculate the electron density as a function of heliocentric distance. Defaults to `utils.N_e_analytical`.
- **verbose** (*bool, optional*) – print additional info at each step. Defaults to True.

Returns

integrated total brightness

Return type

float

```
doppler_dimming_lib.integrals.integrated_I_s(rho: float, _lambda: float, T_e: float, wind_speed: float,
                                             component: int, min_x: float = None, max_x: float =
                                             None, N_e_function=<function N_e_analytical>,
                                             verbose=True) → float
```

Integrate thomson scattering over photospheric wavelenghts, solar disc dimension and line of sight (main integral from Cram 1976).

Parameters

- **rho** (*float*) – heliocentric distance in solar radii
- **_lambda** (*float*) – wavelenght in angstrom
- **T_e** (*float*) – electron temperature
- **W** (*float*) – wind speed in km/s
- **component** (*int*) – polarized component to compute. With respect to solar limb, 0 is radial, 1 is tangent, 2 is pb (=tangent-radial, Cram 1976) and 3 is the sum (total brightness)
- **min_x** (*float, optional*) – integration limit for LOS away from observer in solar radii. Defaults to 10.
- **max_x** (*float, optional*) – integration limit for LOS towards observer in solar radii. Defaults to 10.
- **N_e_function** (*callable, optional*) – function used to calculate the electron density as a function of heliocentric distance, in cm⁻³. Defaults to `utils.N_e_analytical`.
- **verbose** (*bool, optional*) – print additional info at each step. Defaults to True.

Returns

integrated emission in erg cm⁻² A⁻¹ s⁻¹ sr⁻¹

Return type

float

1.1.6 doppler_dimming_lib.inversion module

```
doppler_dimming_lib.inversion.invert_temperature_batch(codex_images: ndarray, wind_speed_pos:
                                                         ndarray | float, densities_dc: ndarray,
                                                         helio_distances_dc: ndarray, pid: list) →
                                                         ndarray
```

Get the temperature by inverting CODEX images ratio.

Parameters

- **codex_images** (*np.ndarray*) – images from CODEX measurements. Should be a [4, m, n] array containing 4 filters measurements [T1, S1, T2, S2] (in order) of shape [m, n].
- **wind_speed_pos** (*np.ndarray*) – [m, n] array or float containing the wind speed intensity in the POS for each pixel of the CODEX measurements. If a float is provided, homogeneous wind speed is assumed.
- **densities_dc** (*np.ndarray*) – datacube of shape [l, m, n] containing electron densities in cm⁻³. The first index contains values along the LOS for each pixel, and POS is assumed to be at `int(l/2)`.
- **helio_distances_dc** (*np.ndarray*) – datacube of shape [l, m, n] containing heliocentric distances of each density datacube entry in solar radii. The first index contains values along the LOS for each pixel, and POS is assumed to be at `int(l/2)`.

- **pid** (*list*) – id used for printing progress.

Returns

temperatures resulting from inversion in the POS, in Kelvin.

Return type

np.ndarray

doppler_dimming_lib.inversion.invert_temperature_parallel(*codex_images: ndarray, wind_speed_pos: ndarray, densities_dc: ndarray, coordinates_dc: ndarray, total_processors: int = None*) → ndarray

Get the temperature by inverting CODEX images ratio. This version first splits the domain for X processors and then applies invert_temperature_batch to each subdomain, before merging the result.

Parameters

- **codex_images** (*np.ndarray*) – images from CODEX measurements. Should be a [4, m, n] array containing 4 filters measurements [T1, S1, T2, S2] (in order) of shape [m, n].
- **wind_speed_pos** (*np.ndarray*) – [m, n] array or float containing the wind speed intensity in the POS for each pixel of the CODEX measurements. If a float is provided, homogeneous wind speed is assumed.
- **densities_dc** (*np.ndarray*) – datacube of shape [l, m, n] containing electron densities in cm⁻³. The first index contains values along the LOS for each pixel, and POS is assumed to be at int(l/2).
- **coordinates_dc** (*np.ndarray*) – datacube of shape [l, m, n] containing coordinates of each density datacube entry in solar radii. The first index contains values along the LOS for each pixel, and POS is assumed to be at int(l/2).
- **total_processors** (*int, optional*) – number of processors for parallelization. This number should be tweaked to avoid asymmetric division of the domain which can increase the computation time. If set to None, 4 processors are used. Defaults to None.

Returns

temperatures resulting from inversion in the POS, in Kelvin.

Return type

np.ndarray

1.1.7 doppler_dimming_lib.metis module

doppler_dimming_lib.metis.Ne_exponential_fit(*x: float, b: ndarray*) → float

Fitted ne based on metis pB.

Parameters

- **x** (*float*) – heliocentric radial distance in R_{sun}
- **b** (*np.ndarray*) – coefficients array

Returns

electron density at a given heliocentric radial distance

Return type

float

`doppler_dimming_lib.metis.Te_pos_from_ne(filename, image_side)`

`doppler_dimming_lib.metis.datacube_from_map(ne_map: GenericMap, coefficients: ndarray) → ndarray`

`doppler_dimming_lib.metis.density_dc_from_ddt(filename: str, size_in_pixel: int) → ndarray`

Get the electron density datacube from fitted metis values.

Parameters

- **filename** (*str*) – fits filename of the metis VL pb.
- **size_in_pixel** (*int*) – output datacube size

Returns

electron density datacube of size [size_in_pixel, size_in_pixel, size_in_pixel]

Return type

np.ndarray

`doppler_dimming_lib.metis.geom_factor_function(r: float) → float`

Geometric factor function from Van de Hulst.

Parameters

r (*float*) – heliocentric distance in solar radii

Returns

computed geometric factor

Return type

float

`doppler_dimming_lib.metis.get_3d_param(param: str, pix_to_rsun: float, coefficients: ndarray, pixel_pos: list, sun_center_pos: list, z_pix: float) → float`

Returns the electron density or temperature from a Metis map fitted with DDT. For the electron temperature calculation see `utls.T_e_analytical`.

Parameters

- **param** (*str*) – parameter to calculate, either Ne or Te
- **map_pos** (*sunpy.map.Map*) – map containing the parameter and metadata in sunpy format
- **coefficients** (*np.ndarray*) – [8, 360] array containing the fitted coefficients in the first axis.
- **pixel_pos** (*list*) – [y, x] pixel coordinates at which to calculate the parameter.
- **sun_center_pos** (*list*) – [y, x] pixel coordinates of the sun center. Provided by the user to avoid useless calculations.
- **x_pos** (*float*) – distance in pixels along the line of sight at which to calculate the parameter.

Returns

electron density at set location

Return type

float

`doppler_dimming_lib.metis.get_ne_map(filename)`

`doppler_dimming_lib.metis.get_pix_to_rsun(input_map: GenericMap) → float`

Returns the platescale in pixel/Rsun

Parameters

input_map (*sunpy.map.Map*) – map for which to calculate the ratio

Returns

pixel/Rsun platescale

Return type

float

`doppler_dimming_lib.metis.read_sav_model(filename: str) → list`

Reads a sav idl file and returns the parameters list.

Parameters

filename (*str*) – path to idl save file (.sav)

Returns

list of 3d array containing the red parameters. In order: positions array, sun radial distances, sun latitude angle, densities, electron temperatures, proton temperatures, solar wind speed.

Return type

list

`doppler_dimming_lib.metis.resample_ddt_ne(new_dimension, ne_header, ne_data, ne_polar_data)`

1.1.8 doppler_dimming_lib.parallelization module

`doppler_dimming_lib.parallelization.batch_2d_array(input_array, tot_nprocs)`

`doppler_dimming_lib.parallelization.batch_3d_array(input_array, tot_nprocs)`

`doppler_dimming_lib.parallelization.batch_arrays(input_list, tot_nprocs)`

`doppler_dimming_lib.parallelization.main()`

`doppler_dimming_lib.parallelization.merge_2d_batched_array(input_array)`

1.1.9 doppler_dimming_lib.utils module

`doppler_dimming_lib.utils.N_e_analytical(r: float) → float`

Returns the electron density as a radial function from sun center.

Parameters

r (*float*) – radial distance in R_sun

Returns

electron density in cm-3

Return type

float

`doppler_dimming_lib.utils.T_e_analytical(rho: float, N_e_function=<function N_e_analytical>, **kwargs) → float`

Returns an analytical electron temperature as a function of `doppler_dimming_lim.utils.N_e_analytical` from Lemaire and Stegen 2016

Parameters

- **rho** (*float*) – heliocentric distance in solar radii

- **N_e_function** (*callable*) – density as a function of heliocentric distance (in sr). Use a polynomial to interpolate data if necessary

Returns

electron temperature profile

Return type

float

doppler_dimming_lib.utils.get_pix_to_rsun(*input_map*)

doppler_dimming_lib.utils.get_sun_center_from_map(*map*)

Returns the coordinates of the pixel containing the sun center. Use this instead of CRPIX!!!!

Parameters

map (*sunpy.map.Map*) – map containing the image and reference frame

Returns

coordinate (x, y)

Return type

list

doppler_dimming_lib.utils.timeit(*func*)

Wrapper for timing functions execution times. Prints the difference between starting and ending time.

Parameters

func (*callable*) – function of which to measure the execution time

1.1.10 Module contents

AAAAAAAAAAAAAAAAAAAAAAAAA

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

d

- `doppler_dimming_lib.c_libraries`, 1
- `doppler_dimming_lib.c_libraries.c_lib`, 1
- `doppler_dimming_lib.codex`, 2
- `doppler_dimming_lib.data`, 1
- `doppler_dimming_lib.datacubes`, 5
- `doppler_dimming_lib.integrals`, 5
- `doppler_dimming_lib.inversion`, 6
- `doppler_dimming_lib.metis`, 7
- `doppler_dimming_lib.parallelization`, 9
- `doppler_dimming_lib.spectra`, 2
- `doppler_dimming_lib.spectra.spectra`, 2
- `doppler_dimming_lib.utils`, 9

B

`batch_2d_array()` (in module `doppler_dimming_lib.parallelization`), 9
`batch_3d_array()` (in module `doppler_dimming_lib.parallelization`), 9
`batch_arrays()` (in module `doppler_dimming_lib.parallelization`), 9

C

`convolve_codex_filter()` (in module `doppler_dimming_lib.codex`), 2

D

`datacube_from_map()` (in module `doppler_dimming_lib.datacubes`), 5
`datacube_from_map()` (in module `doppler_dimming_lib.metis`), 8
`density_dc_from_ddt()` (in module `doppler_dimming_lib.metis`), 8
`doppler_dimming_lib`
 module, 10
`doppler_dimming_lib.c_libraries`
 module, 1
`doppler_dimming_lib.c_libraries.c_lib`
 module, 1
`doppler_dimming_lib.codex`
 module, 2
`doppler_dimming_lib.data`
 module, 1
`doppler_dimming_lib.datacubes`
 module, 5
`doppler_dimming_lib.integrals`
 module, 5
`doppler_dimming_lib.inversion`
 module, 6
`doppler_dimming_lib.metis`
 module, 7
`doppler_dimming_lib.parallelization`
 module, 9
`doppler_dimming_lib.spectra`
 module, 2
`doppler_dimming_lib.spectra.spectra`

module, 2

`doppler_dimming_lib.utils`
 module, 9

G

`geom_factor_function()` (in module `doppler_dimming_lib.metis`), 8
`get_3d_param()` (in module `doppler_dimming_lib.metis`), 8
`get_c_library()` (in module `doppler_dimming_lib.c_libraries.c_lib`), 1
`get_ne_map()` (in module `doppler_dimming_lib.metis`), 8
`get_pix_to_rsun()` (in module `doppler_dimming_lib.metis`), 8
`get_pix_to_rsun()` (in module `doppler_dimming_lib.utils`), 10
`get_R_as_spline()` (in module `doppler_dimming_lib.codex`), 2
`get_spectrum_from_txt()` (in module `doppler_dimming_lib.spectra.spectra`), 2
`get_sun_center_from_map()` (in module `doppler_dimming_lib.utils`), 10
`get_T_from_R()` (in module `doppler_dimming_lib.codex`), 3
`get_T_W_from_codex()` (in module `doppler_dimming_lib.codex`), 3

I

`I_dl_omega_dphi()` (in module `doppler_dimming_lib.integrals`), 5
`I_s_lambda()` (in module `doppler_dimming_lib.integrals`), 5
`integrated_I_s()` (in module `doppler_dimming_lib.integrals`), 5
`invert_temperature_batch()` (in module `doppler_dimming_lib.inversion`), 6
`invert_temperature_parallel()` (in module `doppler_dimming_lib.inversion`), 7

M

`main()` (in module `doppler_dimming_lib.paralleliza-`

tion), 9
map_from_ddt() (in module doppler_dimming_lib.datacubes), 5
merge_2d_batched_array() (in module doppler_dimming_lib.parallelization), 9
module
 doppler_dimming_lib, 10
 doppler_dimming_lib.c_libraries, 1
 doppler_dimming_lib.c_libraries.c_lib, 1
 doppler_dimming_lib.codex, 2
 doppler_dimming_lib.data, 1
 doppler_dimming_lib.datacubes, 5
 doppler_dimming_lib.integrals, 5
 doppler_dimming_lib.inversion, 6
 doppler_dimming_lib.metis, 7
 doppler_dimming_lib.parallelization, 9
 doppler_dimming_lib.spectra, 2
 doppler_dimming_lib.spectra.spectra, 2
 doppler_dimming_lib.utils, 9

N

N_e_analytical() (in module doppler_dimming_lib.utils), 9
Ne_exponential_fit() (in module doppler_dimming_lib.metis), 7

R

read_sav_model() (in module doppler_dimming_lib.metis), 9
resample_ddt_ne() (in module doppler_dimming_lib.metis), 9

S

set_codex_transmittancies() (in module doppler_dimming_lib.codex), 3
set_spectrum_level() (in module doppler_dimming_lib.spectra.spectra), 2
simulate_codex_chunk() (in module doppler_dimming_lib.codex), 4
simulate_codex_filter() (in module doppler_dimming_lib.codex), 4
simulate_codex_images() (in module doppler_dimming_lib.codex), 4
simulate_codex_measure() (in module doppler_dimming_lib.codex), 4

T

T_e_analytical() (in module doppler_dimming_lib.utils), 9
Te_pos_from_ne() (in module doppler_dimming_lib.metis), 7
timeit() (in module doppler_dimming_lib.utils), 10